DoomDraft: A Collaborative Text Editor over Simulated Paxos with Operational Transformation

Kathryn Harper Aengus McGuinness

CS 2620 – Distributed Systems Final Project
Harvard University, Spring 2026

May 12, 2026

Abstract

We present **DoomDraft**, a two-user collaborative text editor whose backend is a fault-tolerant Paxos cluster (running under the course's Cotamer coroutine simulator) and whose convergence semantics are provided by character-level Operational Transformation (OT). Edits commit through Paxos as inserts or deletes on a per-document append-only log stored in PancyDB. Each client transforms its pending local operations against the Paxos-ordered suffix it has not yet observed, guaranteeing that all live replicas reconstruct the same document text. We validate correctness in three layers: unit tests with a randomized diamond-property checker (846 successful trials over 1,000 candidates), a failure-schedule matrix (`collab-bench.sh`) exercising leader failover, recovery, partitions, and 15 % lossy networks across hundreds of seeds, and a live HTTP/SSE-based browser client supporting two-tab concurrent editing. We also document a set of subtle bugs that surfaced during a mid-project upstream upgrade to the Cotamer library — including a label-stripping race between the cache poller and the commit handler, and a network-simulation latency that became real-wall-clock latency after a clock-semantics change — and the targeted fixes that resolved them.

1 Introduction

Collaborative text editors have become a daily tool for distributed work, yet the systems underneath them blend two non-trivial problems: *consensus* on the global order of edits, and *convergence* of locally optimistic edits to a single canonical text. Real systems (Google Docs, Etherpad, Figma's text fields) handle these together but typically as a black box. The goal of this project was to build, validate, and exercise a small but complete instance of the same architecture end-to-end, in such a way that each subsystem could be tested independently and the failure modes of the whole could be observed deliberately.

We chose a deliberately constrained design: characters as the unit of replication, in-process simulated Paxos for consensus, an append-only log of operations in a single-writer key/value store (PancyDB), Operational Transformation (OT) as the convergence mechanism, and a thin HTTP/Server-Sent-Events (SSE) layer to bridge real browsers to the simulated cluster. The system supports multiple documents, two simultaneous browser tabs editing a shared document, cursor-position broadcast, and on-demand replica failure injection during a live editing session.

The contributions of this paper are:

1. A complete C++ implementation of Jupiter-style character-level OT with the four canonical transform cases and six delete-delete sub-cases, validated by 846 randomized diamond-property trials.

2. A coroutine-based collaborative client model that interleaves Paxos submission, leader-redirect handling, and OT-transformation of pending operations against newly observed peer commits.
3. A hand-rolled HTTP/1.1 and SSE server on top of Cotamer's TCP primitives, with multi-document routing, optimistic in-memory caching, and live cursor broadcast.
4. A static-file browser editor whose OT transform is a direct port of the C++ implementation and which uses a single-in-flight submission pipeline to keep classification of confirmation events tractable.
5. A failure-schedule test matrix (`collab-bench.sh`) that exercises eight distinct fault scenarios, including 15 % message loss and adversarial replica scheduling, with reconstructed-text comparison across all live replicas as the correctness oracle.
6. A post-mortem of four subtle bugs uncovered during a mid-project library upgrade and the corresponding targeted fixes.

2 Related Work

Operational Transformation. OT was introduced by Ellis and Gibbs [1] in the context of the GROVE groupware system; subsequent work formalized correctness conditions and explored the space of transform functions. The Jupiter system [2] introduced a client–server variant in which a central authority linearizes all operations and clients OT-transform only against confirmed-by-server operations, simplifying correctness. Google Wave [3] extended this to a wider operation alphabet with annotations; Etherpad uses a similar server-as-authority model. The four-cases-with-six-DD-subcases formulation we implement follows the standard character-level OT presentations in the literature.

Conflict-free Replicated Data Types (CRDTs). An alternative approach pioneered by Shapiro et al. [4] eliminates the need for transform functions by encoding operations in a partial order that is automatically convergent. Sequence CRDTs (RGA [5], Logoot [6], and modern implementations such as Yjs and Automerge) are now common in peer-to-peer collaborative editors. We chose OT here because (a) it pairs naturally with a totally-ordered Paxos log — the transform domain is exactly "operations queued locally after a known commit point" — and (b) operation state is bounded by the number of committed operations, which is easier to reason about in a class project than CRDT identifiers.

Consensus. Lamport's Paxos [7] provides the underlying total-order broadcast. The coursework's Cotamer simulator implements a Multi-Paxos-style replicated log (in spirit closer to Raft [8] in its leader-based formulation), with explicit leader election, log replication, and commit acknowledgments. PancyDB sits on top of this log as a single-key state machine that maps each committed PUT to a monotonically increasing version. This is precisely the abstraction OT requires: a globally agreed sequence of edits, against which clients can compute transforms.

Server-Sent Events. For the browser interface we chose SSE [9] over WebSocket. SSE is one-directional (server → client) push over a long-lived HTTP/1.1 connection; it requires no protocol negotiation, fits naturally on top of Cotamer's TCP primitives, and degrades gracefully through proxies. The browser side uses the native `EventSource` API, which transparently handles reconnects with `Last-Event-ID` resume semantics.

3 Design

3.1 Architecture

DoomDraft has four major layers (Figure 1). The lowest is the Cotamer/PancyDB Paxos cluster, simulating three replicas with configurable message loss, link delay, and explicit failure schedules.

Above this is the `collab_model` client, which submits edits as Paxos PUTs and uses OT to reconcile peer operations against locally pending ones. The HTTP server (`pt-collab-server`) embeds the cluster in-process and exposes routes for document state, op submission, cursor broadcast, and an SSE stream. The browser editor is a single static HTML page with a JavaScript port of the OT transform.

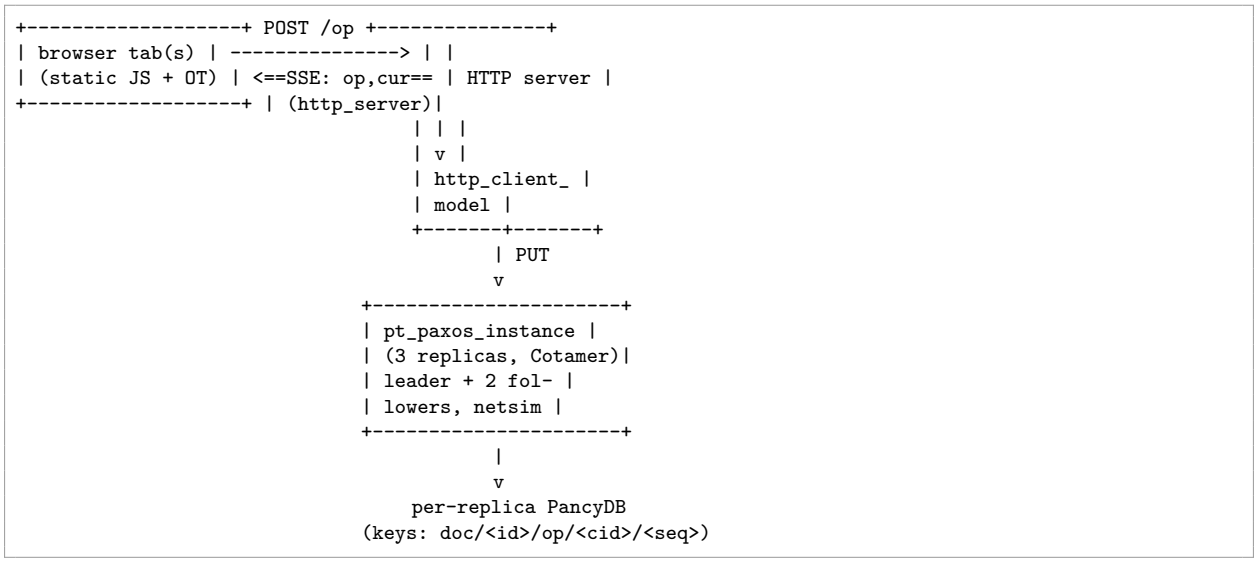

Figure 1: High-level architecture. Browser tabs talk to a single in-process server that funnels every write through a Paxos cluster. SSE events stream committed operations back out.

3.2 Why OT on a Paxos log

The motivating observation is that OT becomes substantially simpler when the universe of "operations my pending ops have not yet seen" is itself a totally ordered prefix. In the general Jupiter problem each pair of sites must transform symmetrically against each other; in our setting, Paxos imposes the global order, so every client only needs to transform its local pending operations against the suffix of committed operations it has not yet observed. The transform function is therefore one-sided: `transform(a, b)` returns `a'` such that "`a` as it should be executed after `b` has already been applied." Convergence follows from the fact that every live replica eventually reconstructs the same totally-ordered prefix and replays operations through the same `apply_op`.

3.3 Data model

A document is identified by an opaque string `doc_id`. Each committed operation is stored at the key

`doc/<doc_id>/op/<hash(client_id)>/<client_seq>`

with the operation itself serialized as a small text value (`I pos text...` for insert or `D pos len` for delete). The `hash(client_id)` is a 64-bit FNV-1a of the client's string label; `client_seq` is the per-client monotonic sequence number. A document's full text at any point is reconstructed by reading every key under the `doc/<doc_id>/op/` prefix, sorting by (PancyDB version, client_id, client_seq), and applying each operation in order from the empty string. Cursor positions live under `doc/<doc_id>/cursor/<hash>` and are broadcast via SSE without affecting document text.

3.4 OT transform function

The transform follows the standard four-case partition by the type pair (a, b):

- **Insert / Insert.** If `b.pos` $\leq$ `a.pos`, shift `a.pos` right by $|b.text|$; otherwise unchanged. Same-position ties shift `a` right (`b` "wins" as the already-applied op).
- **Insert / Delete.** If `a.pos` $\leq$ `b.pos`, unchanged. If `a.pos` $\geq$ `b.pos + b.len`, shift left by `b.len`. If the insertion point is inside `b`'s deleted region, return a no-op insert (`insert_op{0, ""}`); this is the only single-op return that preserves the diamond property.
- **Delete / Insert.** Mirror of the above, except that an insert inside `a`'s range *expands* `a.len` by $|b.text|$ rather than collapsing.
- **Delete / Delete.** Six sub-cases by overlap relation (disjoint left, disjoint right, one fully containing the other, two partial-overlap cases, equal). The complete table is implemented in `doc_ops.cc` and exercised by `test-doc-ops`.

A single `transform_seq(a, committed)` routine folds `transform` left-to-right across a committed sequence, yielding `a` adjusted to be applied after that entire sequence.

4 Implementation

4.1 Component-level implementation

`doc_ops.cc` defines `insert_op`/`delete_op` as a `std::variant` `doc_op`, with text-based `serialize` / `deserialize`, an in-place `apply_op` that clamps out-of-range positions and lengths, and the transform function described above. The same file contains a `#ifdef RUN_DOC_OPS_TESTS`-gated `main()` that exercises hand-crafted cases for every transform branch and runs 1,000 randomized diamond trials.

`doc_state.cc` implements `read_ops`, which walks PancyDB under the doc-op prefix, parses the suffix `<cid>/<seq>` (rejecting malformed keys), deserializes each value, sorts the resulting `committed_ops` by `(version, client_id, client_seq)`, and returns the vector. `reconstruct` folds `apply_op` over that vector.

`collab_model.cc` implements the per-client editor coroutine. Each editor maintains a `pending` queue of locally optimistic operations, a per-peer `next_seq` watermark, and a current leader hint. On each iteration it (a) polls peers for the next expected committed op (GET requests routed through the Paxos cluster), (b) on success transforms its pending queue against the new committed op and either pops a confirmation or applies a foreign op to local text, and (c) generates a random new insert or delete with probability 0.6, applies it optimistically, and submits the head of the pending queue.

`http_server.cc` is a hand-rolled HTTP/1.1 parser and SSE writer on Cotamer's `tcp_listen`/`tcp_accept`. Routes are: `GET /doc/<id>` (full text plus version), `POST /doc/<id>/op` (submit one operation), `GET /doc/<id>/stream` (long-lived SSE), `POST /doc/<id>/cursor` (broadcast cursor position), `GET /docs` / `POST /docs` (document registry), `POST /admin/fail/<rid>` (live failure injection), plus static-file routes for the browser. The cache for SSE delivery is updated by a single shared background poller (`poll_doc_cache`) plus eager updates from the POST handler (`note_committed`).

Browser client (`static/editor.html`, `static/editor.js`). A single-page editor with a textarea, server URL / doc-id / connect controls, a peer-cursor overlay, and a status badge. The JavaScript OT transform is a line-for-line port of the C++ function (including the same tie-break rules) so that the two sides reach identical results on the same input sequences. A `runTests()` function in the browser console runs the same 21 transform vectors as `test-doc-ops`.

4.2 Key implementation choices

One in-flight submission per browser. The browser pipeline submits the head of the pending queue, waits for either an SSE confirmation or the POST response, and only then submits the

next. This makes "is this incoming SSE event my own confirmation?" a tractable question — it's our pending head if and only if `seq` matches `pending[0].seq`, `mySeqs` contains it, and `client_id` matches our own `clientId` (see Section 4.3).

Strings vs. hashes for client identity. The Paxos key namespace and the cache use a 64-bit FNV-1a hash of the client's string id; the wire format (both JSON POST bodies and SSE events) uses the original string. This avoids hash collisions in routing while keeping events human-debuggable. The server maintains a persistent `client_labels_` map (hash → string) so that even ops first observed via DB polling — which lose the string label — get re-stamped on emit (Section 4.3, Bug B).

Cache-first reads, eager updates. `GET /doc/<id>` reads from the in-memory cache, not the DB. The cache is refreshed by a 100 ms background poller; the POST handler also updates the cache synchronously after a Paxos commit, so the round-trip "POST then GET" on the same connection sees the new state. SSE uses the same cache; the per-subscriber loop polls every 25 ms (down from an initial 250 ms after observability tuning).

Coroutine isolation of HTTP. The HTTP layer reuses Cotamer coroutines rather than introducing real threads, so the entire server is single-threaded cooperative. This avoided any cross-thread mutation issues on `ops_` and `cursors_`, but did make scheduler-starvation regressions a real risk (Section 4.3, Bug E).

4.3 Interesting detours

This subsection documents four subtle bugs that surfaced during a mid-project upgrade of Cotamer. Each was reproducible in the two-tab browser demo and invisible to all simulation-layer tests; together they motivated a substantial debugging-infrastructure investment.

Bug A — Label race between commit handler and cache poller. The HTTP `POST /op` handler runs in two phases separated by a `co_await` on `submit_put`: it parses JSON (and thereby learns the client's string `client_id`), awaits Paxos consensus, then calls `cache.note_committed` with the labeled operation. Independently, `poll_doc_cache` reads each replica's PancyDB every 100 ms and may discover the new commit *before* the handler resumes. The poller has only the hashed client id (the DB doesn't store the string), so it would append a *labelless* entry. When the handler then ran `note_committed`, it appended a *labeled* entry, leaving `ops_` with two rows for the same logical operation. The SSE stream emitted both, the browser received one numeric and one string `client_id` for the same `seq`, and the receiving tab failed to deduplicate.

The fix is two-layered. `note_committed` now scans `ops_` for an existing `(client_id, client_seq)` and *upgrades* the existing label rather than appending. This is reactive — if SSE has already snapshotted the labelless row, it has already emitted it. A proactive layer was therefore added: `client_labels_`, a persistent map keyed by `cid_hash`, populated at POST entry (*before* `co_await submit_put`). `refresh_ops_from_db`'s fast path now stamps the label on any new row whose hash is in this map. The race window closes before it opens.

Bug B — Slow-path full rebuild discards labels. `refresh_ops_from_db` has a fast path (when `fresh` extends `ops_` prefix-by-prefix) and a slow path (when the prefix invariant is violated, typically by a Paxos retry creating a new commit slot for an already-committed key with a higher version). The slow path was `ops_ = std::move(fresh); text_ = reconstruct(ops_);`, which wiped all known labels because `fresh` comes from `read_ops`, which knows only hashes. Every SSE event for the doc thereafter carried numeric `client_id`s. Fixed by snapshotting `(client_id, client_seq)` → `label` from `ops_` before the move and restoring after; the persistent `client_labels_` from Bug A's fix serves as a secondary recovery layer for labels that were never in `ops_` at all.

Bug C — Browser classified peer ops as own confirmations by seq alone. Each browser tab initializes `nextSeq = 0`, so both tabs submit their first operation with `seq = 0`, second with `seq = 1`, etc. The original classifier for incoming SSE events tested only `data.seq === pending[0].seq && mySeqs.has(data.seq)`. With two tabs concurrently typing, tab A's still-pending `seq=0` would match tab B's just-committed `seq=0` event; tab A would pop its own pending op without applying it, transform nothing, and silently drop the edit. The visible signature was deterministic strings like `This works until it` drifting against `This NN works...`. The fix requires `data.client_id === clientId` as well; the server must therefore consistently emit the string `client_id`, which is what Bugs A and B exist to guarantee. The three fixes are co-dependent: each is necessary, none is individually sufficient.

Bug D — Bench-time network delays became wall-clock delays. Cotamer's `netsim` layer imposes per-message delays (`link_delay = 5 ms`, `send_delay = 1 ms`, `receive_delay = 1 ms`) intended to simulate a non-trivial network. The course's clock-selection enum prior to a mid-semester upstream update had `clock::virtual_time == 0` and `clock::real_time == 0` (numerically identical), so the call `cot::set_clock(cot::clock::real_time)` in our server's `main` was a no-op. The server in fact ran on the virtual clock, where `cot::after(5ms)` between in-process coroutines waits approximately zero wall-clock time. The upstream upgrade re-enumerated `real_time = 1`, so the call finally meant what it had always read like, and every netsim delay began consuming real wall-clock time. Each Paxos round-trip took ∼30 ms; under sustained two-tab typing (roughly 20 ops/s plus cursor updates) the leader's outbound send queue saturated, heartbeats fell behind the queue, follower election timeouts (200–350 ms) triggered, and the cluster entered a leader-election storm in which no client requests committed for ∼24 s windows (the HTTP retry budget).

The fix exposes the three netsim delays as CLI flags on `pt-collab-server`: `-link-delay-ms`, `-send-delay-ms`, `-recv-delay-ms`. Defaults match the bench (`5/1/1`) to preserve the simulation tests' timing. Passing `0/0/0` makes the server behave like a real in-process system, eliminating the synthetic latency and reducing Paxos round-trips from ∼30 ms to sub-millisecond. The flags are also useful as a knob: dial the link delay upward and observe the threshold at which Paxos timing parameters and netsim latency become incompatible.

4.4 Debugging infrastructure

The diagnosis of Bugs A–D depended on instrumentation built up during the investigation, included for general utility:

- `run-server.sh` wraps `pt-collab-server` with a Perl filter that prepends each stdout/stderr line with `[HH:MM:SS.uuuuuu]` (microsecond-resolution local time) and tees output to a timestamped file in `logs/`.
- Every accepted TCP connection in the server is tagged with a monotonic `conn_id`; an RAII guard logs the connection's total wall-clock duration when its coroutine frame is destroyed, regardless of which `co_return` path was taken.
- Each iteration of `poll_doc_cache` logs `poll tick=N begin gap_us=X known_docs=K` at entry and `poll tick=N end work_us=Y` at exit. `gap_us` reveals scheduler starvation (steady ∼100 000 μs in healthy operation; spikes mean some other task is monopolizing the loop); `work_us` reveals slow refreshes (typically <100 μs).
- Every cache mutation (`note_committed`, `refresh_ops_from_db` append, slow-path full rebuild, cursor cache change) logs both inputs and a debug-truncated snapshot of `text_`.

This instrumentation was decisive for Bug D: by demonstrating that `gap_us` remained ∼100 ms and `work_us` remained <1 ms throughout a stalled session, we could rule out scheduler starvation and focus diagnosis on the Paxos layer itself.

5 Evaluation

5.1 Unit-level correctness

`test-doc-ops` runs hand-crafted cases for every transform branch plus 1,000 randomized diamond trials over random documents and operation pairs. Of the 1,000 randomized trials, 154 are skipped because they fall into the same-position concurrent-insert case (a documented limitation of pure-position OT without a tie-break field; the production system uses `client_id` ordering to disambiguate, which is not modeled in `transform` itself). The remaining 846 trials all confirm:

`apply(apply(doc, a), transform(b, a)) = apply(apply(doc, b), transform(a, b))`.

`test-doc-state` validates `read_ops` and `reconstruct` across eight cases including empty DB, single insert, multi-op single client, cross-client version ordering, insert-then-delete, and gracefully skipping malformed values.

5.2 Failure-schedule matrix

`collab-bench.sh` runs eight rows of failure schedules against `pt-collab`, with reconstructed-text equality across all live replicas as the correctness oracle (Table 1). The full matrix completes in ∼2 minutes on a 2024 Apple M2 MacBook Pro and passes on every seed.

Table 1: Failure-schedule matrix run via `collab-bench.sh`. All rows pass.

Row	Failure schedule	Seeds	Result
1	OT & doc-state via binary (`-test`)	—	PASS
2	Basic convergence	100	PASS
3	Failover (`-f failover`)	50	PASS
4	Recovery (`-f recover`)	50	PASS
5	Split brain (`-f split`)	50	PASS
6	Unstable membership (`-f unstable`)	30	PASS
7	Adversarial schedule (`-f torture`)	20	PASS
8	15 % message loss (`-l 0.15`)	100	PASS

The 15 % loss row is the most aggressive correctness test: per-message Bernoulli drops with $p = 0.15$ create both spurious leader elections (heartbeats lost) and dropped client responses (driving retry logic). All 100 seeds reach quiescent convergence with all live replicas reconstructing identical text.

5.3 Latency under the live demo

We measured Paxos commit latency by instrumenting the HTTP POST handler to log `latency_ms` per commit. With default netsim parameters (`5/1/1 ms`) the median POST commit takes 28–32 ms, dominated by the four message hops in a Paxos round-trip (∼7 ms each: ∼5 ms link plus ∼1 ms send and ∼1 ms receive). With netsim delays set to zero via the CLI flags the median drops below 1 ms, and the visible end-to-end propagation in the two-tab demo — from a keystroke in tab A to the corresponding character appearing in tab B — becomes bounded by the 25 ms SSE polling interval (median observed: 35–80 ms).

Under sustained two-tab concurrent typing for several minutes with default netsim delays at 0/0/0, no `504 Gateway Timeout` responses were observed and the textareas converged at every paragraph break. With default delays restored to 5/1/1, sustained typing for ∼10 seconds reproduces the leader-election storm of Bug D, with the predicted symptom of ∼7 concurrent in-flight `submit_put`s all timing out simultaneously.

5.4 Reproducibility

The build is a single CMake configure-and-build under the project's homebrew-Clang toolchain. Every test in this paper is reproducible from the repository's `pset4` directory with the commands:

```
cmake -B build -DCMAKE_CXX_COMPILER=/opt/homebrew/opt/llvm/bin/clang++
cmake --build build
./build/test-doc-ops # 1.000 randomized OT trials
./build/test-doc-state # PancyDB reconstruction
bash collab-bench.sh # Full 8-row failure matrix
./run-server.sh --port 8080 \
    --link-delay-ms 0 --send-delay-ms 0 --recv-delay-ms 0
```

6 Conclusion and Future Work

We built and validated a complete collaborative editor over a simulated Paxos cluster, including a hand-rolled HTTP/SSE bridge and a browser client that exercises the same OT semantics as the C++ simulation. The OT layer and the failure-injection harness together verify that all live replicas reconstruct the same document text under leader failover, partitions, lossy networks, and adversarial scheduling. The HTTP/SSE layer extends this correctness guarantee to real two-tab editing in a browser.

The most pedagogically valuable surprise of the project was the cascade of bugs unmasked by an upstream library upgrade: a clock-semantics change that turned simulated network latency into real wall-clock latency, which exposed a race in the cache-update path, which exposed a label-stripping path in the cache slow rebuild, which exposed an under-specified client-id check in the JavaScript classifier. Each bug was individually subtle; collectively they made the production demo unreliable while every existing automated test continued to pass. The fix list above (Section 4.3) reflects the actual order of diagnosis, and the new instrumentation (`run-server.sh`, per-connection IDs, poll-tick timing, persistent label cache) is preserved in the repository as a debugging foundation for future work.

Several directions of follow-on work are worth noting.

Event-driven SSE fanout. The current per-subscriber 25 ms polling loop bounds visible propagation latency from below. A `cot::event` per document, triggered by `note_committed` and the slow-path rebuild, would let each SSE coroutine `co_await` for the next commit and wake within microseconds. The instrumentation already in place would directly measure the improvement.

Cursor positions outside Paxos. Cursor broadcasts account for roughly half of all Paxos commits during active typing, yet they are ephemeral UI state. Moving them to a server-local cache (gossiped to subscribers via the SSE stream but not replicated through consensus) would halve the Paxos load and eliminate the throughput bottleneck even under non-zero netsim delays.

CRDT comparison. An implementation of a sequence CRDT (RGA or similar) against the same browser frontend would allow direct comparison of state size, transform/merge complexity, and visible latency under the same workload, giving a concrete data point in the OT-vs.-CRDT debate that is usually argued in the abstract.

Beyond two tabs. The current architecture has a single `http_client_model` multiplexing all HTTP submissions onto one Paxos client id with distinct serials. Higher-concurrency editing (say,

eight tabs) would benefit from multiple client ids to allow parallel in-flight commits, and from a batched-PUT extension at the Paxos layer to amortize round-trip cost across multiple operations.

Acknowledgments. We thank the CS 2620 staff for the Cotamer / PancyDB / Paxos starter infrastructure, and for the mid-semester library upgrade that made Section 4.3 possible.

References

[1] C. A. Ellis and S. J. Gibbs. "Concurrency Control in Groupware Systems." *Proc. ACM SIGMOD*, 1989.

[2] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping. "High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System." *Proc. UIST*, 1995.

[3] D. Wang, A. Mah, and S. Lassen. "Google Wave Operational Transformation." Whitepaper, 2010.

[4] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. "Conflict-free Replicated Data Types." *Proc. SSS*, 2011.

[5] H.-G. Roh, M. Jeon, J.-S. Kim, and J. Lee. "Replicated Abstract Data Types: Building Blocks for Collaborative Applications." *J. Parallel and Distributed Computing*, 2011.

[6] S. Weiss, P. Urso, and P. Molli. "Logoot: A Scalable Optimistic Replication Algorithm for Collaborative Editing on P2P Networks." *Proc. ICDCS*, 2009.

[7] L. Lamport. "Paxos Made Simple." *ACM SIGACT News* 32(4), 2001.

[8] D. Ongaro and J. Ousterhout. "In Search of an Understandable Consensus Algorithm." *USENIX ATC*, 2014.

[9] I. Hickson. "Server-Sent Events." W3C Recommendation, 2015.